

location counter and pointer arithmetics

SEGMENT data use32 class = data:

a db 1,2,3,4 → a string
l db \$-a ; l=4

↑ location counter

l_0 db \$-data ; l_0=5 → syntax error

l_1 db a-data

l_2 dw data-a

00400100

→ "Expression is not simple or relocable".

(I dk what value to give to data)

assembler ✓ (because they start at the same address?)

linker X → linker error.

db a-\$; -5 = FBh

c equ a-\$; c = -6 = FAh

d equ a-\$; d = -6 = FAh

e db a-\$; e = -6

} the same bc. no other variables have been declared.

x dw x ; x = 0710h → 7 bytes starting from the beginning of the data segment.

x db x

x_1 dw x_1 ; x_1 = 0910h

db l-a ; 04

db a-l ; FC (-4)

db [\$-a]

db [a-\$]

$$\text{offset}(l) = \frac{00400000 - 00401004}{\text{FFFFFFFF}}$$

→ syntax error (a memory constant is not constant or compatible at assembly time)

li equ li ; = 0 NASH

li equ li-a ; = 0 NASH, offset(a) = 0 !!!
 (0-0)

g dw c-2 ; c = -6 ; (-2) → -6-2 = -8

b dd a-start → syntax error (expression is not simple or relocable)

dd start-a ✓

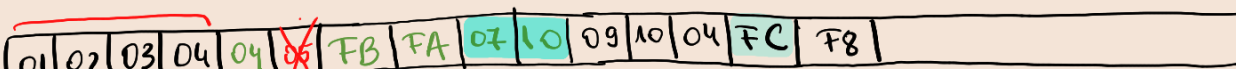
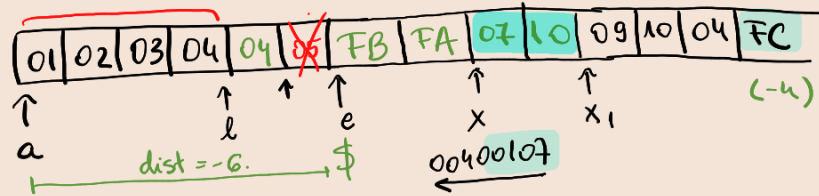
↓
 FAR POINTER

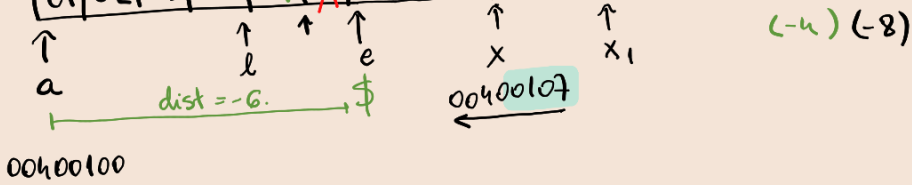
SUBTRACTION

↓
 a defined here, but

start defined somewhere else.

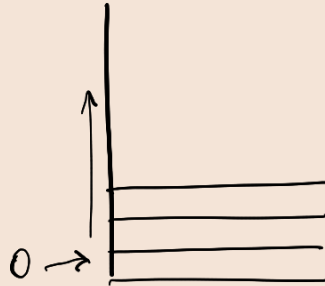
dd start1-start ✓ → 2 labels defined in the same data segment and result is a scalar type!





Initial data segment value: DS: 00401000
CS: 00402000
ES:

Flat memory:



segment code $use32$ class = code:
start:

`mov ah, l1 ; ah = 0`

`mov bh, c ; bh = -6 = FA`

`mov ch, l ; ch = ? error (obj. format can handle only 16 or 32 byte relocation)`

`mov ch, l-a ; ch = 4.`

`mov ch, [l-a] ; mov ch, byte ptr DS : [4] !! access violation.`

`mov cx, l-a ; cx = 4.`

`mov cx, [l-a] ; error`

`mov cx, $-a ; error (invalid operand type)`

↳ cannot be evaluated at assembly time.

`mov cx, $$-a ; error (invalid operand type)`

`mov cx, a-$` ✓ (a defined somewhere else, $\$$ defined in the current segment)

`mov cx, $-start` ✓

`mov cx, start-$` ✓

`mov cx, a-start`

`mov ah, a+b` ✓ (no pointer arithmetic)
 ↳ no pointer addition
 ↳ yes scalar addition
 ↓

$$a+b = (a - \$\$) + (b - \$\$)$$

`mov ch, [a+b] ; INVALID effective addrs.`

↓
 pointer addition

var1 dd a+b ; syntax error (cannot assign ...)
(expr. is not simple or relocatable)

expression of type label1 - label2 (labels in either code/data seg.)
↓

syntactically accepted in NASM.

→ both defined in the same segment)

→ even if label1 belongs to a $\neq$ segment than label2,
the expression is accepted by NASM and the type assigned
to the result is a POINTER and NOT a SCALAR!

Example: mov bx, [$(r_3 - r_2) \pm (r_1 - r)$] ✓
scalar

Indirect addressing: mov bx, [$r_1 - r_2 - r$] ; NASM $r_1 - (r_2 + r)$
mov bx, $r_2 + r_1 - r$; ✓
(scalar) ↓
invalid operand type!

Jump instructions

JMP near / far.

segment data use32 class = data:

aici dd here ; "here" is a label

segment code use32 class = code:

mov eax, [aici] ; EAX = content of "aici"; offset of "here".

mov ebx, aici ; EBX = offset of "aici"; content of "here"
= 00401000h

jmp [aici] ; jump to the address indicated by the value of var. "aici".

$\equiv \times$

✓ jump to address given by label "here" ← jmp here ✓ → jmp eax ✓

← jmp [ebx]

jump to address stored in mem. loc. having the address of the contents of EBX. ✓

× jmp [eax] ← jmp [here] ← jmp aici (same thing as jmp ebx)

↳ jmp cs:EIP

offset (ici) from DS.
↓
jump to ? some instruction
segmentation fault
⇒ "access violation".

here:

Interaction between the implicit association rules of an offset with the corresponding segment register

- Example ↴
- `jmp [var_mem] → jmp DWORD PTR DS: [00401704]`
NEAR JUMP to this offset ↴

Ex: `jmp [ebx]` ;

`jmp DWORD PTR DS: [EBX]`
NEAR JUMP to this offset ↴

Ex: `jmp [ebp]`

stack segment!
↓
`jmp DWORD PTR SS: [EBP]`
NEAR JUMP to this offset ↴

Example: ↴
Ex: `jmp here ; jmp [SHORT] 00402024`

- What is a **FAR JUMP** ?

`jmp DS: [EBX]` → not a far jump

FAR will only be the ADDRESS from which the OFFSET will be taken where the NEAR JUMP will be.

`jmp [ss: ebx+12]`
`jmp [ss: ebp+12] ≡ jmp [ebp+12]` } NEAR JUMP in the same segment

If I want a FAR jump (different seg.), we must explicitly specify this by means of the FAR operator ⇒ a jump to a far address.

`jmp far [ebx+12] ; CS:EIP ← far address (48 bits = 6 bytes)`

`jmp far [DS:EBX+12] ; CS:EIP ← far address (48 bits = 6 bytes)`

CS: 62C2h

EIP: 61527A9Bh

9B7A5261C265
String of bytes at offset
DS: EBX+12.

⇒ `mov CS:EIP, [adr_far]`

`jmp label, NEAR jump.`

Example: segment data ...
a db 1,2,3,4

start3:

`mov ebx, eax` ✓

segment code use 32 class = code:

start:

`mov edx, eax`

`jmp 00403000 ← jmp start2 ✓ (near jump)`

`jmp 00401004 ← jmp start3 ✓ (near jump)`

`jmp far start2` ✗

`jmp far start3` ✗

⋮

`add eax, 1`

final:

`push dword 0`

`call [exit]`

segment code1 use 32 class = code:

start2:

`mov eax, ebx`

`push dword 0`

`call [exit]`